

separates the logic of the application as well as abstracts the underlying implementations of the required functionality.

Although they are not a new concept in programming, services introduce a host of beneficial functionalities to angular. Dependency injection is one such feature that we will see and use extensively in most of our examples later with services. Services in angular as crucial in helping to keep the application modular and maintainable. Not only is an incorrectly defined service more difficult to maintain, it can even be less unit testable leading to more bugs in the code. Services, therefore, are a critical part of the angular application.

It is perhaps important to note that all angular services are only available in the scope that they are defined in. We will delve more on services and scopes later in the chapter.

Structure

A service in angular consists of a typescript class that consists of one or more functions or values that are used by other components / services in the application.

It is also recommended to annotate a service class with `@Injectable` which tells angular that the metadata of this class allows injection of values into it. Although non-mandatory, it is recommended to do this for code clarity and future proofing.

Thus, the syntax of a Service looks somewhat like this:

```
• import { Injectable } from '@angular/core';
@Injectable()
export class MyService {
  serviceMethod(): void {}}
```

Notice that the class contains a single method “serviceMethod” that is currently stubbed.

The objective here is not to implement a functionality but to show how the implementation of the “serviceMethod” is abstracted into this service class.

Any other component/service who is using the “MyService” class need simply use its serviceMethod function without caring about how it is being implemented. All reusable and non - view related logic should always be abstracted into services in this manner. This service can then be used in any other component in the application by referring to its instance.